Name: Aditya Vikas Borkar

College Name: MIT Arts, Commerce and Science College Alandi Pune

Course: SY M.Sc. Data Science

Company/Organization: PERSEVEX

Location: Bengaluru

Role: Data Science Intern

Project : 2nd

Assigned Date: 03/02/2026 Completion Date: 05/03/2026

Project Title:

Resume parsing and candidate ranking for recruiter

Objective

The primary objective of this project is:

- To automatically extract text from multiple PDF resumes.
- To extract text from a given job description.
- To compute similarity between resumes and the job description.
- To rank candidates based on their relevance score.
- To export ranked results into a downloadable CSV file.

This system acts as a basic **automated resume screening and candidate ranking engine** for recruitment use cases.

Tools & Technologies Used

Programming Environment

- Python
- Google Colab

Libraries

- pdfminer.six → PDF text extraction
- pandas → Data handling & ranking
- scikit-learn
    - TfidfVectorizer → Text vectorization
    - cosine_similarity → Similarity computation

- zipfile → Dataset extraction
- os → File management
- google.colab.files → File upload & download

Methodology

- The workflow follows a structured NLP-based document similarity pipeline:
- Upload resumes + job description
- Extract text from PDFs
- Convert documents to TF-IDF vectors
- Compute cosine similarity
- Rank resumes by match score
- Export results

Output Screenshots (What You Would See)

Since this is a notebook-based project, the outputs would appear like this:

Output 1: Resume Processing Count

```
resume_folder = os.path.join(
    extract_path,
    "HireAMLE",
    "dataset",
    "trainResumes"
)

resume_texts = []
resume_names = []

for file in os.listdir(resume_folder):
    if file.endswith(".pdf"):
        file_path = os.path.join(resume_folder, file)
        try:
            text = extract_text(file_path)
            if text and len(text.strip()) > 0:
                resume_texts.append(text)
                resume_names.append(file)
        except:
            continue

print("Total Resumes Processed:", len(resume_texts))
```

Total Resumes Processed: 90

Output 2: Job Description Loaded

```
job_description_path = os.path.join(
    extract_path,
    "HireAMLE",
    "dataset",
    "Job description.pdf"
)

job_description = extract_text(job_description_path)

print("Job Description Loaded Successfully!")
```

```
Job Description Loaded Successfully!
```

Output 3: Ranked Candidates Table

```
ranking_df = pd.DataFrame({
    "Candidate_Name": resume_names,
    "Match_Score_%": percentage_scores
})

ranking_df = ranking_df.sort_values(
    by="Match_Score_%",
    ascending=False
).reset_index(drop=True)

ranking_df.head(15)
```

	Candidate_Name	Match_Score_%
0	candidate_134.pdf	22.404539
1	candidate_137.pdf	20.327925
2	candidate_063.pdf	20.058371
3	candidate_121.pdf	19.680439
4	candidate_096.pdf	17.653033
5	candidate_093.pdf	17.494831
6	candidate_050.pdf	17.161868
7	candidate_133.pdf	16.903473
8	candidate_034.pdf	16.810700
9	candidate_009.pdf	16.580583
10	candidate_055.pdf	15.673127
11	candidate_113.pdf	15.387083
12	candidate_023.pdf	15.350187
13	candidate_008.pdf	15.276774
14	candidate_089.pdf	15.216431

Output 4: CSV Download Prompt

```
output_file = "candidate_ranking_results.csv"
ranking_df.to_csv(output_file, index=False)

from google.colab import files
files.download(output_file)
```

5 Conclusion

This project successfully implements a **basic AI-driven resume screening system** using classical NLP techniques.

Strengths

✓ Automated PDF resume parsing
✓ TF-IDF-based feature extraction
✓ Cosine similarity ranking
✓ Scalable to large datasets
✓ Simple and computationally efficient